RA23E01 – ROBOT OPERATING SYSTEM

AGILEX SCOUT MINI NAVIGATION USING ROS 2 WITH WHEEL-SLIP SIMULATION AND CORRECTION (SECOND REVIEW)

Submitted By:

SAILAKSHMI BT (2024511019)

BACHELOR OF ENGINEERING
in
ROBOTICS AND AUTOMATION

DEPARTMENT OF PRODUCTION TECHNOLOGY
MADRAS INSTITUTE OF TECHNOLOGY
ANNA UNIVERSITY - CHENNAI 600044

BONAFIDE CERTIFICATE

Certified that this project report “AgileX Scout Mini Navigation using ROS 2 with Wheel-Slip Simulation and Correction (Second Review)” is the Bonafide work of Sailakshmi BT (2024511019) who carried out the project work under my supervision.

SIGNATURE

SUPERVISOR

TABLE OF CONTENTS:

S.No	TOPICS	PAGE
1	**Introduction**	4
2	**Work Done Since the First Review**	4
3	**The robot: drive motors, actuators and sensors**	5
4	**The slip world**	9
5	**The slip controller (slip_control)**	13
6	**The Ground Station: slip panel and direct routing**	20
7	**Algorithms Used**	21
8	**How to Run**	23
9	**Experimental method**	24
10	**Results**	26
11	**Problems solved**	30
12	**Discussion and limitations**	33
13	**Conclusion and future work**	33
14	**Appendices (A and B)**	34

Abstract

This report presents the second review of the AgileX Scout Mini simulation in ROS 2 Jazzy and Gazebo Harmonic: wheel slip on a low-friction floor, a software slip controller, direct shortest-path goal sending in the Ground Station, and a new set of measurements. A second world with three flush patches of friction $\mu = 0.04$ makes the wheels slip; a slip_control node estimates the true ground speed from the IMU, detects slip, limits the wheel command, holds the heading and publishes corrected odometry for the EKF. The report also records the drive motors, actuators and sensors of the real and simulated robot.

Nine planned patrol runs of 20 legs (three per condition) and two extra runs were made. On the normal floor all four runs completed 20 of 20 legs with ground-truth errors up to 0.37 m. On the slip floor with the correction on, 3 of 4 runs completed; without it, 2 of 3 completed and the robot overshot the corridor hub in all three runs. Two route tours showed driven distances within +1.8 % (normal) and +2.2 % (slip floor) of the shortest grid path. All results are from simulation.

How the slip is corrected, in one line: the IMU gives the true ground speed, and when the wheels turn faster than that the controller slows the wheel command to the true speed and corrects the odometry before the EKF uses it.

1. Introduction

The first review presented the Scout Mini navigating a 20 m × 16 m civil facility with six rooms in Gazebo. A SolidWorks model was converted to URDF (28 kg, four driven wheels), the facility and its occupancy map were generated, and Nav2 (EKF, AMCL, Smac 2D planner, regulated pure pursuit) drove the robot through the rooms from a PyQt5 Ground Station. One recorded 23-leg patrol reached 23 of 23 legs with a mean ground-truth error of 0.25 m. In that work the floor always gave good grip, so the robot could not lose traction, and the Ground Station split every click into door-aligned waypoints.

The second review asks what happens when the floor does not give grip and whether the robot still takes the shortest route. A real robot meets wet or polished patches; when a wheel slips, the wheel odometry counts distance that was never travelled and localisation drifts. The objectives of the second review are:

- to add a second world with low-friction patches and to show wheel slip;
- to estimate the true speed, detect slip and correct it (traction limiting, heading hold, odometry correction) in a slip_control node, with a switch to turn the correction off for comparison;
- to show slip in the Ground Station and to send each click as one direct shortest-path goal with pre-emption;
- to record the motors, actuators and sensors of the robot;
- to measure everything again: nine patrol runs, two extra runs and route tours, with each number traced to a log file.

Section 2 lists what changed since the first review. Sections 3 to 6 describe the robot with its Gazebo and RViz2 views, the slip world, the slip controller and the Ground Station; Section 7 gives the algorithms used for slip handling and for goal and mission management; Section 8 explains how to run the simulation; Section 9 gives the method; Section 10 the results; Sections 11 to 13 the problems solved, limitations and conclusion.

2. Work Done Since the First Review

The table compares the two reviews. The robot model, the facility, localisation and the Nav2 stack of the first review are unchanged in kind; the floor, the slip control, the click routing and the measurements are new. The second review asked three questions: (1) does navigation still work when the floor has patches of very low friction; (2) can a software slip controller reduce the damage; and (3) does the robot take the shortest route when the user clicks a goal in another room? While answering them, the Nav2 configuration, the Ground Station and the patrol script were changed, and everything was measured again.

Table 1. Changes between the first and the second review.

Item	First review	Second review
Floor	Uniform floor, μ = 0.25 wheels on good grip	Second world with three flush low-friction patches (μ = 0.04); slip controller added
Click routing	Each click split into door-aligned waypoints	One direct NavigateToPose goal; the Smac 2D planner chooses the shortest path
Pre-emption	Several clicks were needed	route_id counter: a new click drops the old route at once
Goal handling	Goal could be rejected near walls	Goal snapped ≥ 0.55 m from obstacles; goal held within 0.30 m for 6 s counts as reached
Nav2 costs	cost_travel_multiplier 2.0	cost_travel_multiplier 1.0 (planner minimises distance)
Goal tolerance	yaw_goal_tolerance 0.35 rad	yaw_goal_tolerance 3.14 rad (position only; no spin on arrival)
Patrol	23 legs, one recorded run	20 legs (three corridor via-point legs removed, see note below), nine planned runs plus two extra runs
Launcher	Stale slip_control processes could survive between runs	run_civil_nav.sh stops slip_control at start-up and clean-up

Why the patrol has 20 legs instead of 23. The three removed legs were the extra corridor via-points (corridor at the CAD door, hub westbound, corridor at the Materials door) that had been added in the first review so that the robot followed door-aligned lanes. In the second review the click routing was changed to one direct goal per click, with the Smac 2D planner choosing the shortest path (Section 6). The three via-points were therefore no longer needed to steer the robot, and the West-lane waypoints that follow them already lie on the same corridor lane. The three legs were removed; all three rooms and all stations are still visited. The patrol is the same route without those three corridor stops, so its results should not be compared leg-for-leg with the earlier 23-leg run.

3. The robot: drive motors, actuators and sensors

A. Drive system of the real Scout Mini

The values below were read from public manufacturer and reseller pages on 10 November 2026. They differ between sources and none was checked on a physical robot; they are given for comparison with the simulated model, which does not model the motors electrically.

Table 2. Drive system of the real Scout Mini (published values).

Item	Published value	Note
Drive layout	4WD skid-steer, independent drive of each wheel	Turns in place (turning radius 0)
Drive motors	DC brushless, 4 × 150 W	Manufacturer manual; one reseller lists 4 × 250 W hub motors for the standard-wheel version, so the sources disagree
Wheel position sensing	Hall encoder	Listed by one reseller only
Battery	24 V, 15 Ah lithium	Runtime about 3 h unloaded and about 1 h at full payload (reseller figure)
Payload	10 kg over long distances, up to 50 kg for short distances	Manufacturer documentation
Maximum speed	10 km/h (about 2.8 m/s)	Other sources quote 2.7–3.0 m/s
Mass	20 kg (sellers list 20–26 kg)	Depends on configuration
Control input	Not listed in the sources consulted	Simulation: /cmd_vel (geometry_msgs/Twist) from teleoperation or Nav2

B. Actuators in the simulation

Each of the four wheels is driven by a continuous joint (j1 and j2 on the left, j3 and j4 on the right). The Gazebo DiffDrive system turns the velocity command (v, ω) into wheel joint velocities, so the wheels follow the command rigidly: there is no motor current, torque curve, gearbox or battery model. This is why slip has to be created by lowering the floor friction (Section 4) and why the slip controller limits the command instead of the motor current.

Table 3. Actuators of the simulated robot.

Parameter	Value	Source
Wheel joints	j1 and j2 on the left side, j3 and j4 on the right side (as configured in DiffDrive); type continuous	Assem2.SLDASM.urdf
Joint limits	effort 40, velocity 25 rad/s; joint damping 0.1, joint friction 0.1	URDF
Wheel radius / separation	0.08 m / 0.612 m (skid-steer treated as an equivalent differential drive)	DiffDrive plugin
Plugin speed limits	±2.5 m/s linear, ±3.5 rad/s angular	DiffDrive plugin
Plugin acceleration limits	±4.0 m/s² linear, ±6.0 rad/s² angular	DiffDrive plugin
Command timeout	0.5 s (the robot stops if no command arrives)	DiffDrive plugin
Wheel contact	$\mu_1 = \mu_2 = 0.25$; $k_p = 10^6$, $k_d = 100$, min depth 0.001	URDF Gazebo tags
Odometry and joint states	/odom at 50 Hz (odom → base_footprint); /joint_states at 50 Hz	DiffDrive and JointStatePublisher plugins

Examples. For straight driving the wheel angular speed is v/r: 7.5 rad/s at the Nav2 cruise speed of 0.6 m/s and 8.75 rad/s at the smoother limit of 0.7 m/s. In a turn at v = 0.6 m/s and ω = 1.0 rad/s the outer wheels turn at $(v + \omega b/2)/r = 11.3$ rad/s and the inner wheels at 4.0 rad/s. The URDF joint velocity limit of 25 rad/s equals 2.0 m/s at the rim, below the 2.5 m/s plugin limit; whether Gazebo enforces the URDF limit was not tested, and it does not matter for navigation, which is limited to 0.7 m/s.

Command path. Nav2 controller → velocity smoother → /cmd_vel → slip_control → /cmd_vel_safe → ros_gz_bridge → Gazebo DiffDrive → wheel joints. With the correction switched off, slip_control passes the command through unchanged.

C. Sensors in the simulation

All sensors are Gazebo sensors defined in the URDF and bridged to ROS 2 by ros_gz_bridge. Noise values are those written in the URDF.

Table 4. Sensors of the simulated robot.

Sensor	Gazebo type, topic, frame	Rate	Range, field of view, resolution	Noise	Used by
2D LiDAR	gpu_lidar, /scan, lidar_link	20 Hz	720 samples over −π..π (0.5° steps); 0.12–16 m; range resolution 0.01 m	Gaussian, σ = 0.01 m	AMCL, local and global costmaps, SLAM Toolbox (mapping mode)
IMU	imu, /imu, imu_link	100 Hz	3-axis gyroscope and 3-axis accelerometer	Gyro σ = 0.0002 rad/s; accelerometer σ = 0.017 m/s² per axis	EKF (yaw rate); slip_control (acceleration, yaw rate)
RGB-D camera	rgbd_camera, /camera (colour as /camera/image_raw), camera_optical_frame	30 Hz	640 × 480, R8G8B8; field of view 80° (1.396 rad); depth 0.1–10 m	None set in the URDF	Ground Station view only (not used by Nav2)
Wheel odometry	DiffDrive, /odom, odom → base_footprint	50 Hz	Integrates the commanded wheel motion	None (ideal wheels; slip changes the true motion, not the reading)	EKF; slip_control

Why these sensors. The LiDAR gives range data for localisation and obstacle marking; AMCL ignores returns closer than 0.55 m so that the robot body is not matched against the map. The IMU gives the yaw rate (wheel-derived yaw drifts on a skid-steer base; about 60–65 % of the commanded yaw was measured before the IMU was fused) and the accelerations used by the slip estimator. Wheel odometry gives the short-term motion estimate; on a slippery floor it over-reports, which is the error slip_control corrects. The camera is not used for navigation.

Sensor fusion. The EKF runs at 20 Hz in 2D mode: forward speed from wheel odometry (from /odom_corrected when the slip controller is active) and yaw rate from the IMU, publishing odom → base_footprint. AMCL (1000–4000 particles, likelihood-field model, σ_{hit} = 0.20 m) publishes map → odom from /scan and the map.

D. Software nodes and topics

Table 5. Nodes and programs of the system.

Node or program	Package or file	Role
ros_gz_bridge	ros_gz_bridge	Bridges /clock, /scan, /imu, /odom, /camera/image_raw and /joint_states; subscribes to /cmd_vel_safe
robot_state_publisher	robot_state_publisher	Publishes the URDF kinematic tree and wheel joint transforms
ekf_node	robot_localization	Fuses wheel speed and IMU yaw rate; publishes /odometry/filtered and odom→base_footprint
map_server, amcl	nav2_map_server, nav2_amcl	Serve the occupancy map and publish map→odom
planner, controller, smoother, bt_navigator	Nav2	Smac 2D A* planner, rotation-shim and regulated pure pursuit controller, velocity smoother, behaviour-tree navigator
slip_control	assem2_robot/scripts	Slip estimation, traction limiter, heading hold, /odom_corrected
civil_interactive_nav.py	assem2_robot/scripts	PyQt5 Ground Station: camera view, map, click-to-navigate, slip panel, emergency stop
civil_nav2_mission.py, nav_tour_test.py	assem2_robot/scripts	20-leg patrol with ground-truth verification; route-efficiency tour
generate_civil_world.py, generate_slip_world.py	assem2_robot/scripts	Generate the facility world and the slip world

E. Gazebo and RViz2 views

The figures below show the model and the simulation as built and used in the first stage, on the normal floor: the SolidWorks assembly, the facility in Gazebo and an autonomous patrol in Gazebo and RViz2. The Gazebo window could not be captured during the slip runs of this review because the Wayland desktop blocks screen capture (problem 10 in Section 11); the slip floor is therefore shown through Gazebo camera renders, RViz2 window grabs (through XWayland) and Ground Station screenshots in Sections 4 and 6.

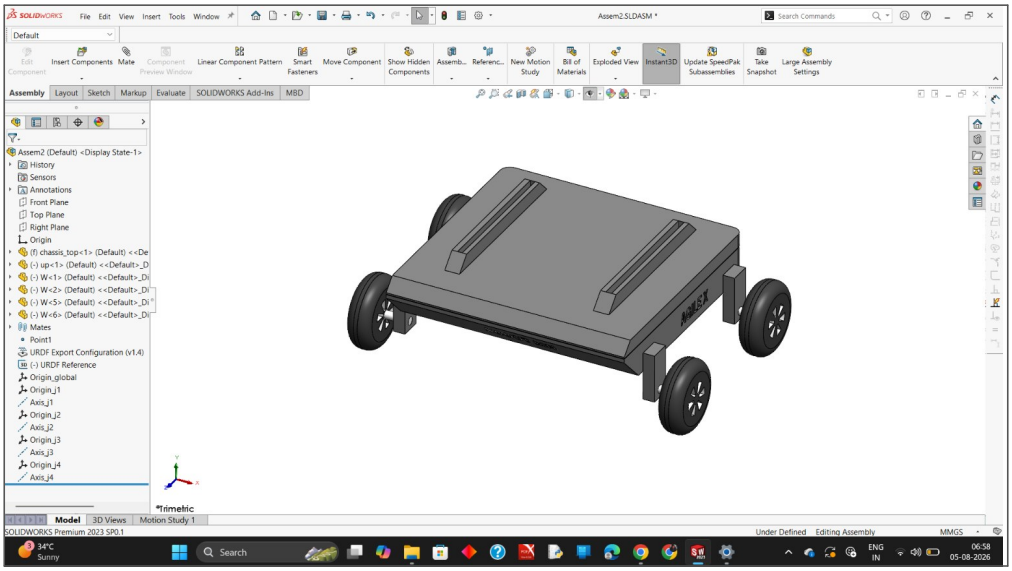


Fig. 1. SolidWorks assembly of the robot (chassis, upper deck, four wheels, URDF export set-up) before conversion to URDF.

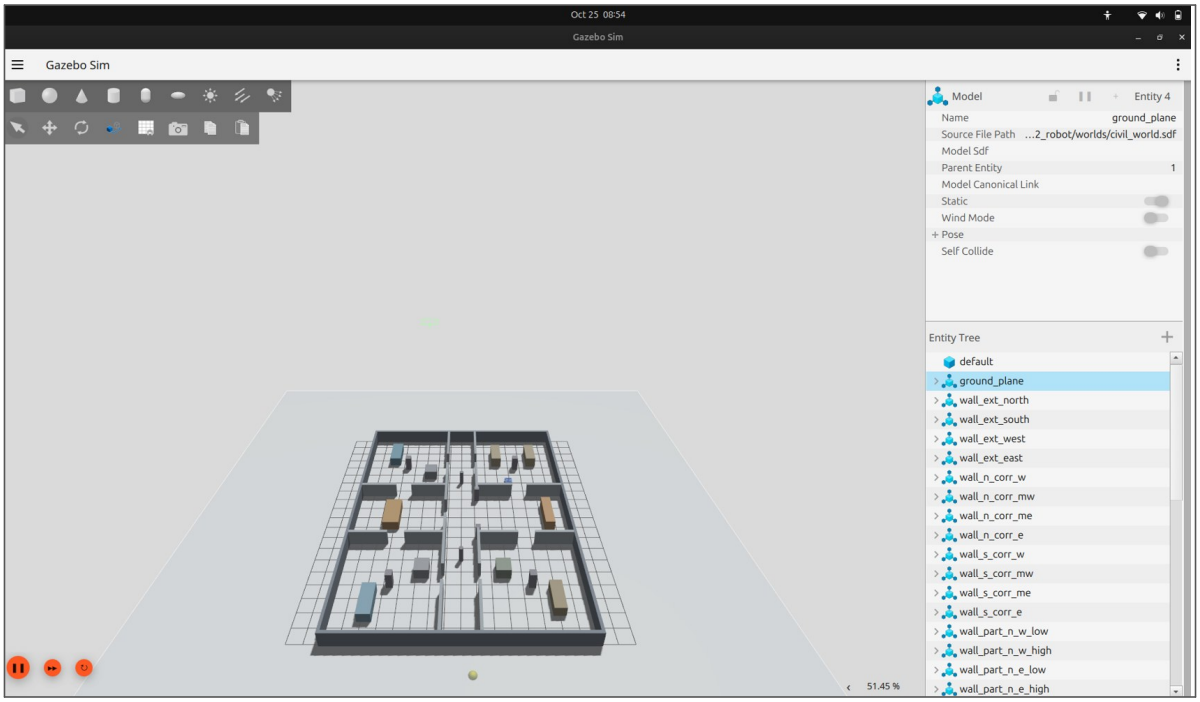


Fig. 2. The civil facility in Gazebo Harmonic with the robot, the entity tree and the world panel.

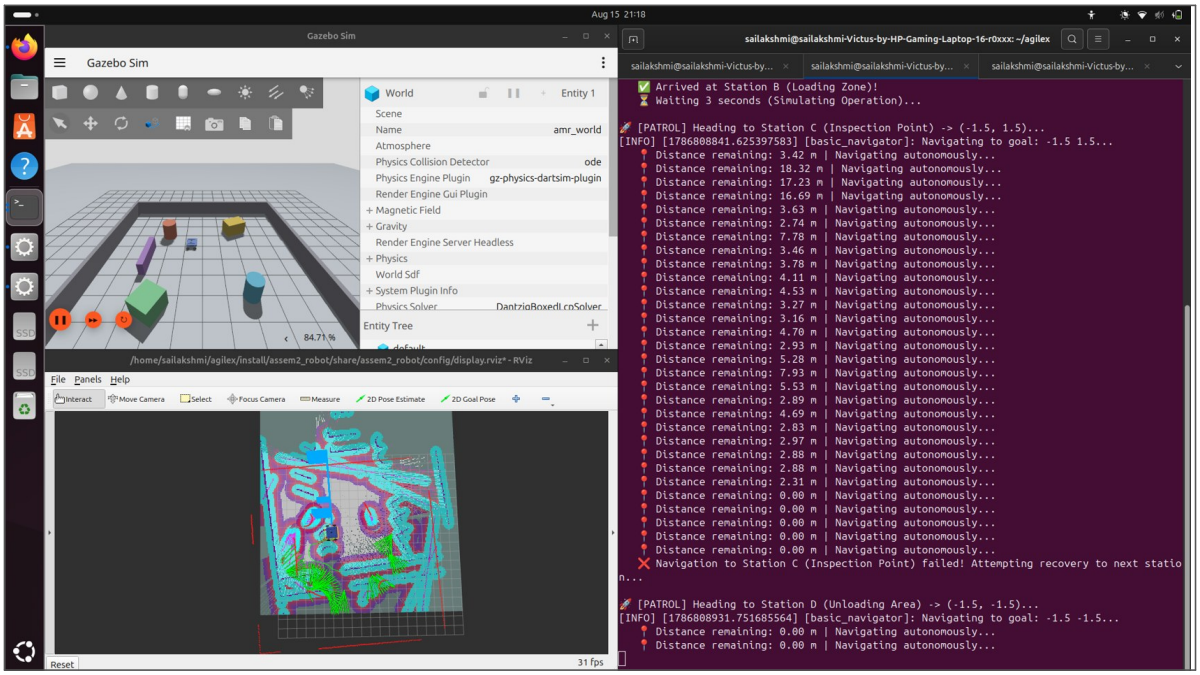


Fig. 3. Autonomous patrol: Gazebo (left), RViz2 (bottom) and the patrol terminal reporting distance remaining.

4. The slip world

A second world, civil_world_slip.sdf, is generated from the civil world by generate_slip_world.py. It is the same facility with three flush low-friction patches. A patch has μ = 0.04, against μ = 0.25 for the wheels, so it can supply only a very small acceleration:

$a_{max} = \mu g = 0.04 \times 9.81 \approx 0.39$ m/s² on a patch, against $0.25 \times 9.81 \approx 2.5$ m/s² that the wheels can be commanded to deliver on dry floor.

Table 6. Low-friction patches of the slip world (verified in civil_world_slip.sdf).

Patch	Centre (x, y) [m]	Size (x × y) [m]	μ
Conference Hall doorway (north)	(0.0, 2.3)	1.2 × 1.8	0.04
East corridor	(1.1, 0.0)	1.0 × 2.4	0.04
West corridor	(−1.1, 0.0)	1.0 × 2.4	0.04

The patches had to be flush with the floor: a first version with raised tiles stopped the robot at the 3 mm edge (problem 1 in Section 11). The generator therefore removes the collision of the ground plane, lays floor tiles around holes of the patch size and fills the holes with the low-friction patches. The corridor patches lie between the corridor hub and the columns and cover the full corridor width, so that turns at the doors happen on dry floor and the robot is already straight on the ice (problem 13). Because the Gazebo wheels follow velocity commands rigidly, slip occurs whenever the commanded acceleration exceeds what the floor supplies. With the correction off, the wheels then turn faster than the body moves (Fig. 11: wheels 0.48 m/s, ground 0.40 m/s, slip 16 %), the odometry counts distance that was never travelled, and the EKF and AMCL believe the robot is farther along than it is.

The four Gazebo views below were rendered from the slip world itself (correction on, robot at the corridor hub). The Gazebo window cannot be captured from the desktop, so virtual camera sensors were placed in the running world (script gz_cams.py) and their images saved; the blue areas are the three low-friction patches.



Fig. 4. Slip world from above, north at the top: the facility with its six rooms, the robot at the corridor hub and the three blue low-friction patches (north doorway above the robot, east and west corridor patches beside it).



Fig. 5. Oblique Gazebo view of the whole slip world.

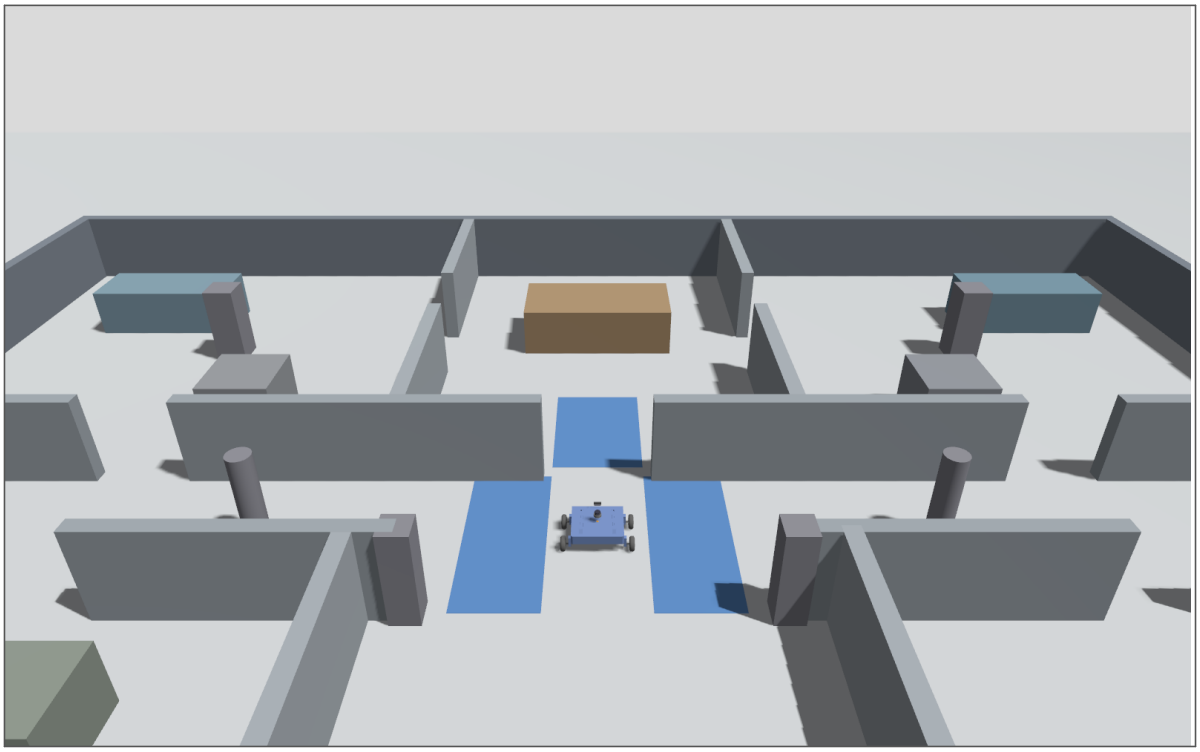


Fig. 6. The corridor hub seen from the south: the robot on dry floor with the three patches around it.

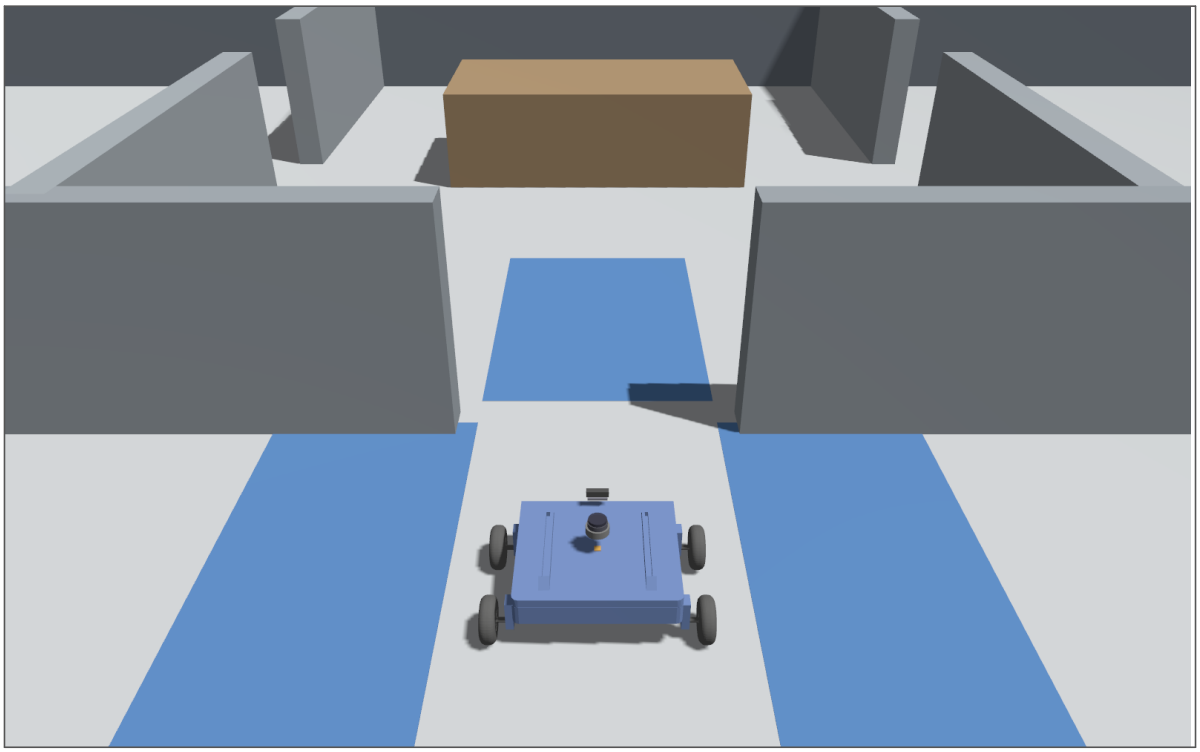


Fig. 7. The Scout Mini model in front of the north doorway patch, with the east and west corridor patches on either side.

RViz2 views of the same slip world are given next. They were grabbed with xwd from the RViz2 window, which runs under XWayland; the windows of Gazebo itself could not be grabbed, which is why its views above come from camera sensors. RViz2 is configured to show the occupancy map and costmaps, the LiDAR scan, the robot model, the plans and the TF frames; in the two views below the walls and obstacles of the map appear as black cells. The slip patches are flush with the floor and carry no map or costmap marking, so they are not visible in RViz2; this is why the slip controller has to detect slip from motion.

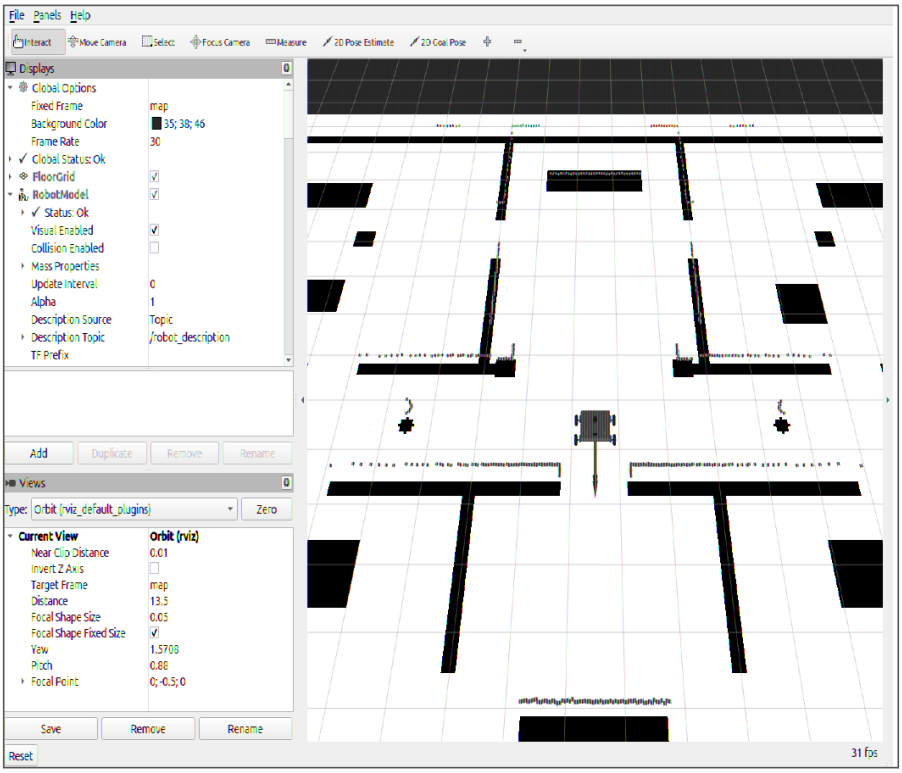


Fig. 8. RViz2 in the slip world at start: robot at the corridor hub, LiDAR scan along the walls and the black occupied cells of the map. The panel shows the display tree and the Orbit view.

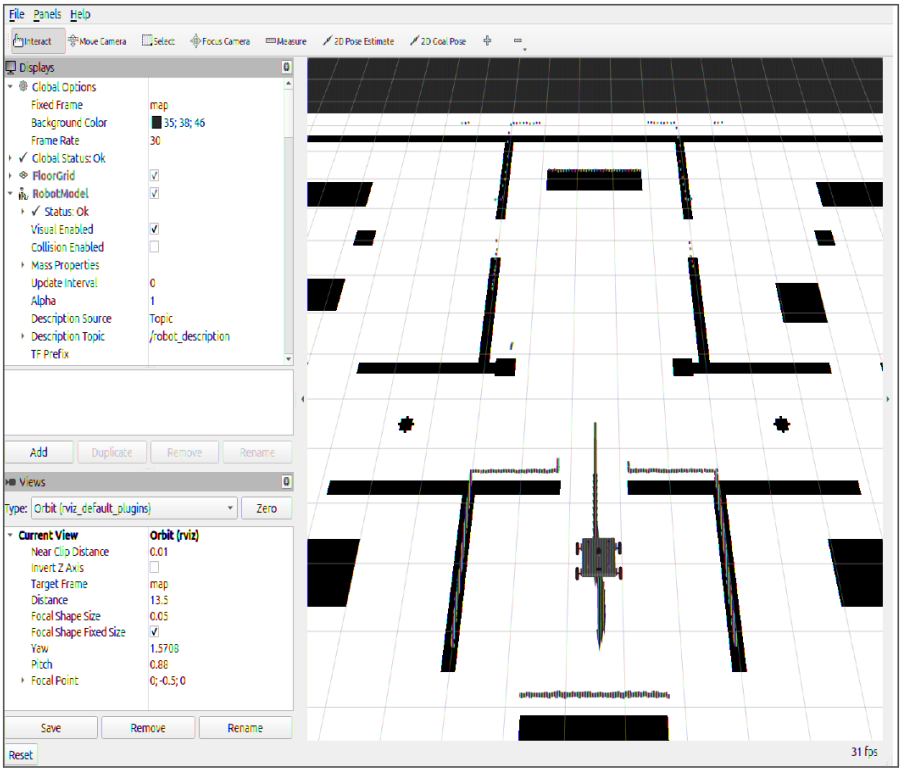


Fig. 9. RViz2 while the robot drove through the north doorway to a goal at (0, 2.6) over the north slip patch; the thin line ahead of the robot is the Nav2 plan. The goal succeeded. /slip/active was sampled a few times during the drive and read false each time, so slip was not seen at those instants; the figure is a view of the setup, not evidence of slip.

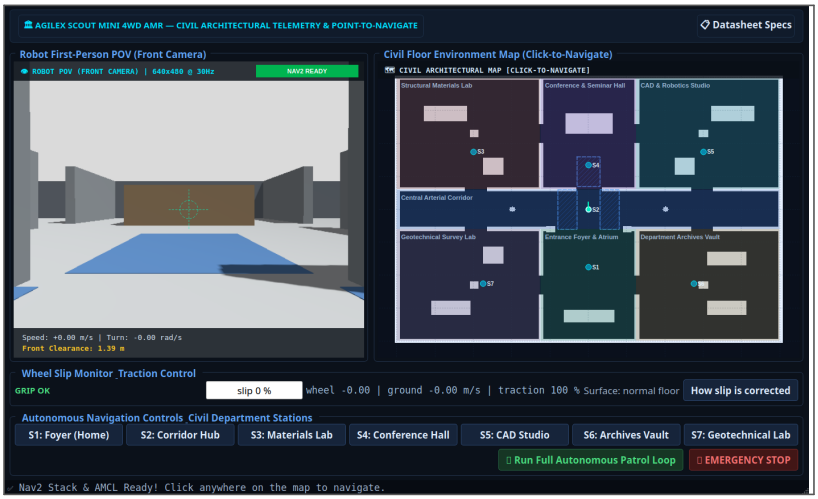


Fig. 10. Ground Station on the slip floor: the blue patch in the Conference Hall doorway is visible in the camera view and as a hatched box on the map (earlier corridor-patch positions).

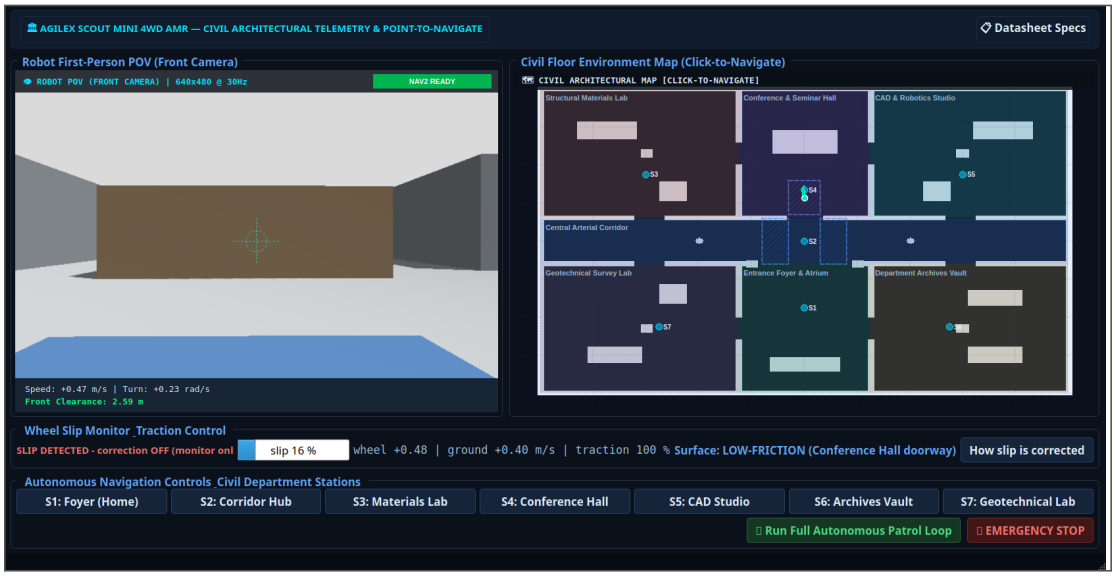


Fig. 11. Correction OFF on the Conference Hall patch: wheels 0.48 m/s, ground speed 0.40 m/s, slip 16 %, traction 100 % (no limiting).

A first straight-line test on an earlier, slipperier floor (μ = 0.02) gave a position error of +1.12 m with the correction off, against +0.04 m with it on.

5. The slip controller (slip_control)

A. Where it sits

How the slip is corrected, in one line: the IMU gives the true ground speed, and when the wheels turn faster than that the controller slows the wheel command to the true speed and corrects the odometry before the EKF uses it.

slip_control (scripts/slip_control.py) sits between Nav2 and the wheels. The bridge subscribes to /cmd_vel_safe instead of /cmd_vel, and the node reads Nav2's /cmd_vel, limits it and publishes /cmd_vel_safe. The EKF no longer reads the raw /odom but the corrected odometry /odom_corrected, and it fuses both the forward and the sideways speed; the IMU yaw rate is used as before. These changes are made in civil_simulation.launch.py, which passes the argument slip_correction. Detection runs at the IMU rate (100 Hz) and the command limiter at 50 Hz.

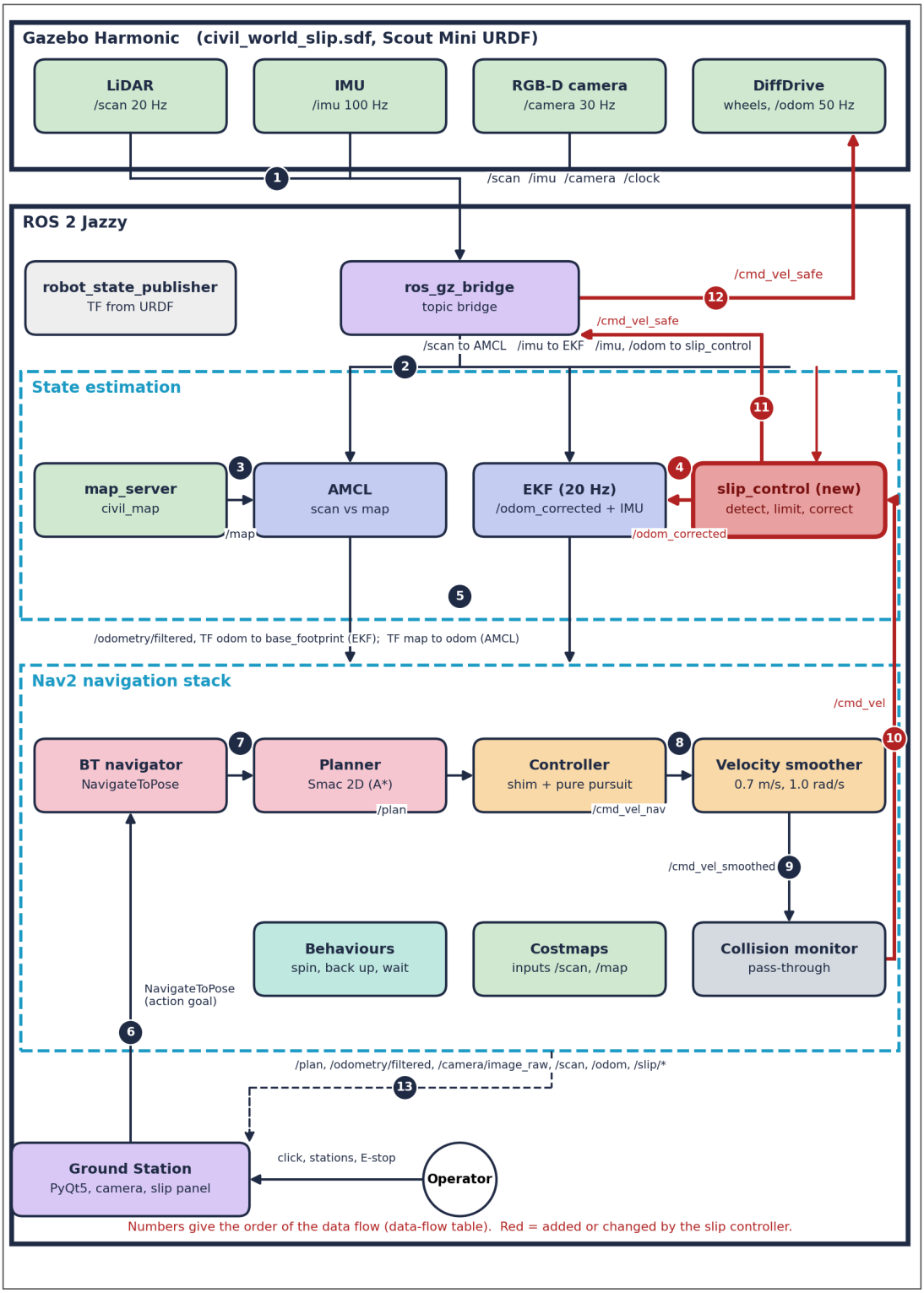


Fig. 12. Complete architecture with the slip controller, redrawn and checked against civil_simulation.launch.py, the Ground Station code and the running ROS graph (red marks what the slip controller changed or added; numbers give the order of the data flow of the table below).

Table 7. Data flow of the architecture with slip control.

Step	From → To	Topic or data	What it does
1	Gazebo sensors → ros_gz_bridge	/scan, /imu, /odom, /clock, /camera/image_raw	Sensors and wheel odometry enter ROS 2 (on the slip floor the odometry over-reports on a patch)
2	Bridge → AMCL, EKF, slip_control	/scan to AMCL; /imu to the EKF; /imu and /odom to slip_control	slip_control receives raw wheel odometry and IMU; the EKF still reads the IMU directly
3	Map server → AMCL	/map (civil_map)	Unchanged
4	slip_control → EKF	/odom_corrected (vx and vy)	New: slip-corrected odometry replaces raw /odom as the EKF wheel input
5	State estimation → Nav2	/odometry/filtered; TF odom→base_footprint, map→odom	The map pose stays close to the true pose
6	Ground Station or mission → BT navigator	NavigateToPose action (RViz2 uses /goal_pose)	A click is one direct goal that cancels the old route
7–9	BT navigator → planner → controller → smoother → collision monitor	ComputePathToPose, FollowPath, /plan, /cmd_vel_nav, /cmd_vel_smoothed	Unchanged (the collision monitor is a pass-through)
10	Collision monitor → slip_control	/cmd_vel	Changed: the command no longer goes straight to the bridge
11	slip_control → ros_gz_bridge	/cmd_vel_safe	New: traction limiter and heading hold act while slip is detected
12	Bridge → Gazebo DiffDrive	/cmd_vel_safe	The bridge subscribes to /cmd_vel_safe
13	Nav2 and slip_control → Ground Station	/plan, /odometry/filtered, /camera/image_raw, /scan, /odom, /slip/*	The slip panel shows slip, ground speed, traction and surface

B. Estimating the true speed and detecting slip

Slip means that the wheel speed is not the speed of the body. The IMU does not depend on the wheels, so it provides an independent measurement. The node integrates the forward IMU acceleration (gravity removed using the IMU orientation) into a ground-speed estimate and blends it with the wheel speed in a complementary filter. The blend is used only when the wheel and the IMU agree about the acceleration, so that a spinning wheel cannot pull the estimate along; a zero-velocity update is applied only when the wheels, the accelerometer and the gyroscope are all quiet; and the estimate is kept below an energy bound. Slip is the difference between wheel speed and estimate.

Table 8. Detection parameters (defaults of slip_control.py).

Parameter	Value	Purpose
Slip enter threshold	$\lvert v_{wheel} - v_{ground} \rvert > 0.15$ m/s for 0.05 s	Declares slip; avoids false alarms from noise
Slip exit threshold	below 0.06 m/s	Clears the slip flag
Fusion gate	0.05 m/s (and acceleration agreement)	Prevents a spinning wheel from corrupting the estimate
Latch breaker	body steady for 0.5 s while slip is still flagged	Resets the estimate to the wheel speed so slip does not stay on
Energy bound	$\lvert v_{est} \rvert \le v_{peak} + 0.05$ m/s; v_{peak} decays at 0.3 m/s²	Stops integration drift from running away

Fig. 13 shows the same data flow as rqt_graph reports it on the running slip world: slip_control sits between the collision monitor (/cmd_vel) and the bridge (/cmd_vel_safe), reads /imu and /odom, and publishes /odom_corrected for the EKF.

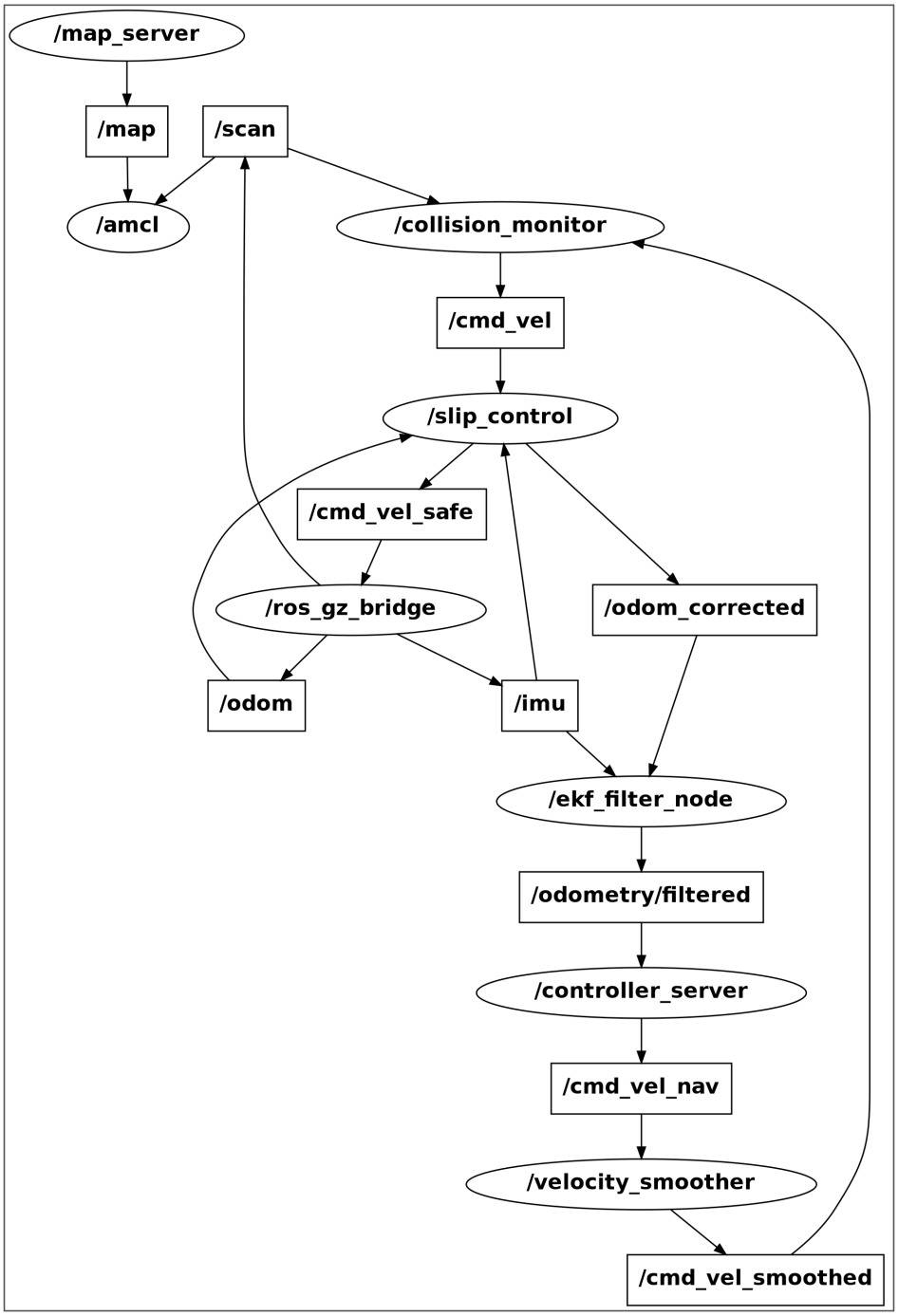


Fig. 13. rqt_graph of the running slip world (Nodes/Topics (all), correction on). The graph was produced by rqt_graph itself from the live ROS 2 system and laid out top to bottom for the page; /tf, action, bond and parameter topics and the Nav2 helper nodes are hidden. The slip path is /cmd_vel → slip_control → /cmd_vel_safe → ros_gz_bridge, and /odom_corrected feeds the EKF.

Table 9. Estimation formulas (as implemented; v_w wheel speed, v_{est} estimated ground speed, a acceleration, G blending weight).

Step	Formula	Meaning
Gravity removal	$a = a_{imu} - g \cdot 2(q_x q_z - q_w q_y)$; $a_{lp} \leftarrow a_{lp} + 0.3\,(a - a_{lp})$	Removes the gravity leaked into the forward axis by pitch and roll, then smooths it
Ground speed	$v_{est} \leftarrow v_{est} + (a_{lp} - a_{bias})\,dt$	Integrates the IMU acceleration; independent of the wheels
Blending (no slip)	$v_{est} \leftarrow v_{est} + \min(3.0\,G\,dt, 1)\,(v_w - v_{est})$; $G = \exp(-(d/0.05)^2) \cdot \exp(-((a_w - a_{lp} + a_{bias})/0.4)^2)$; $d = v_w - v_{est}$	Pulls the estimate to the wheels only while wheel and IMU agree about speed and acceleration
Energy bound	$v_{peak} = \max(\lvert v_w \rvert, v_{peak} - 0.3\,dt)$; $\lvert v_{est} \rvert \le v_{peak} + 0.05$	The body cannot be faster than the fastest recent wheel speed
Slip speed and ratio	$\Delta v = v_w - v_{est}$; $s = \Delta v / \max(\lvert v_w \rvert, 0.2)$, $-1 \le s \le 1$	Example: $(0.35 - 0.19)/0.35 \approx 46$ %, as in the Ground Station of Fig. 16

C. Correcting the slip

Once slip is detected, four actions are taken (Fig. 14):

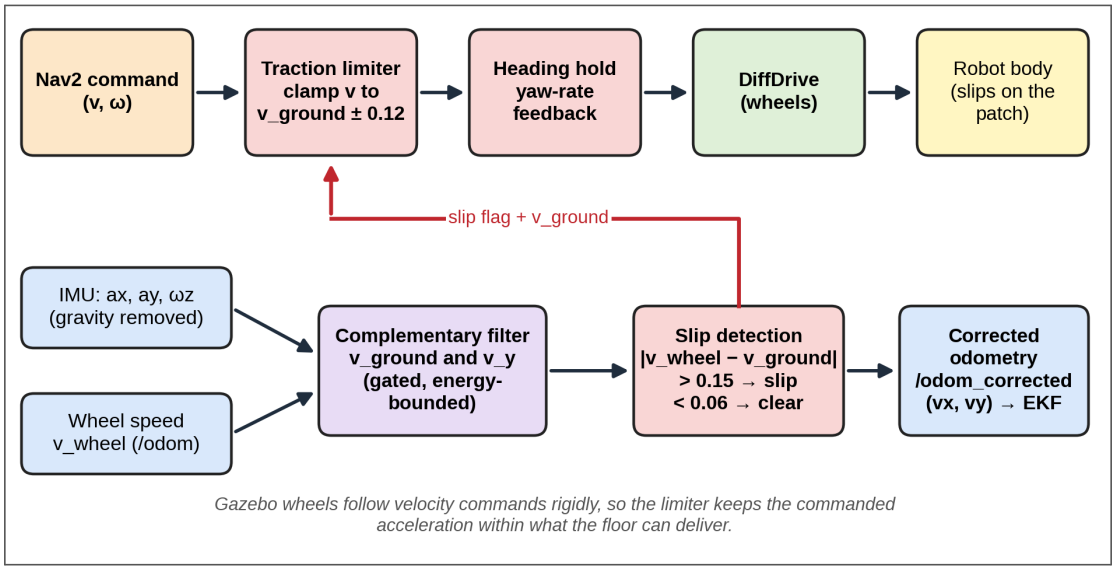


Fig. 14. Inside slip_control: the ground speed is estimated from the IMU and the wheel speed, slip is flagged, the wheel command is limited and corrected odometry is published.

Table 10. The four correction actions.

Action	What it does	Reason
1. Traction limiter	While slipping, and for 0.6 s afterwards, the wheel command is clamped to $v_{ground} \pm 0.12$ m/s and can never exceed the larger of the request and the current output; the angular acceleration limit is halved (6.0 to 3.0 rad/s²); the linear limit stays at 4.0 m/s² (a slip limit of 0.8 m/s² is declared in the code but not used)	Like traction control in a car: less torque lets the wheels grip again
2. Heading hold	The IMU yaw rate is fed back (gain 0.4, limit ±0.6 rad/s)	Uneven grip makes the robot veer; this steers it back
3. Sideways slip	Lateral speed v_y is integrated from the IMU lateral acceleration minus the turning term, and pulled to zero when the tyres grip	Wheel odometry cannot see a sideways slide
4. Odometry correction	/odom_corrected carries the wheel speed with the slip removed (dead-bands 0.04 m/s for v_x and 0.03 m/s for v_y) and the sideways speed; the EKF uses it	The map pose stays close to the true pose, so Nav2 and AMCL keep working

Formulas as implemented: traction limiter $v_{out} = clamp(v_{cmd}, v_{est} - 0.12, v_{est} + 0.12)$; heading hold $\Delta\omega = clamp(0.4(\omega_{cmd} - \omega_{imu}), -0.6, +0.6)$; sideways speed v_y integrates $(a_y - a_{y,bias} - \omega\, v_{est})$; speed correction removes the part of $|\Delta v|$ above 0.04 m/s without changing sign.

D. Switching and monitoring

The correction is switched off with the launch argument slip_correction:=false. The node then still estimates and publishes the slip but does not change the commands or the odometry. It publishes /slip/active, /slip/ratio, /slip/traction_scale and /slip/ground_speed_est, and a latched topic /slip/correction_enabled so that the Ground Station shows which mode is running. The control loop is shown in Fig. 15: the traction limiter acts only while slip is detected and for 0.6 s afterwards, whereas the heading hold and the odometry correction run all the time; the heading hold only adds a yaw-rate correction (at most 0.6 rad/s) and the odometry correction removes only speed differences above the 0.04 m/s dead-band.

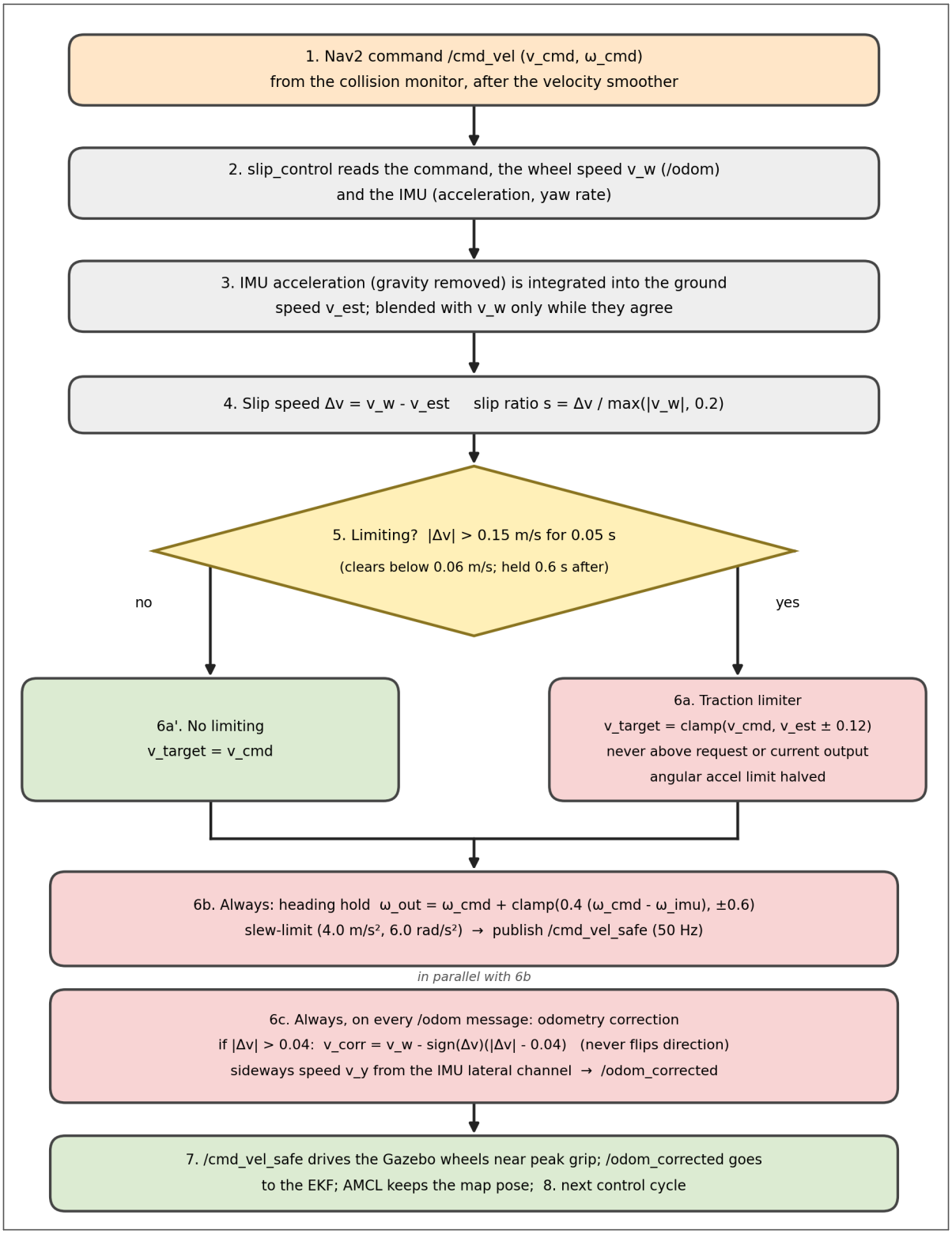


Fig. 15. Flowchart of the control loop with the slip controller, checked against slip_control.py: only the traction limiter depends on the slip decision; the heading hold and the odometry correction run all the time (when the correction is switched on).

6. The Ground Station: slip panel and direct routing

Slip panel. The Ground Station shows a Wheel Slip Monitor: a slip percentage bar, wheel speed against ground speed, the traction scale, the surface under the robot (“normal floor” or “LOW-FRICTION” with the zone name) and a button explaining the correction. The label reads “GRIP OK”, “SLIP DETECTED – traction control + odometry correction” or, when the correction is off, “SLIP DETECTED – correction OFF (monitor only)”. The three patches are drawn on the map as blue hatched boxes. The first version of the label said “correction ACTIVE” even when it was off; the latched topic /slip/correction_enabled fixed this.

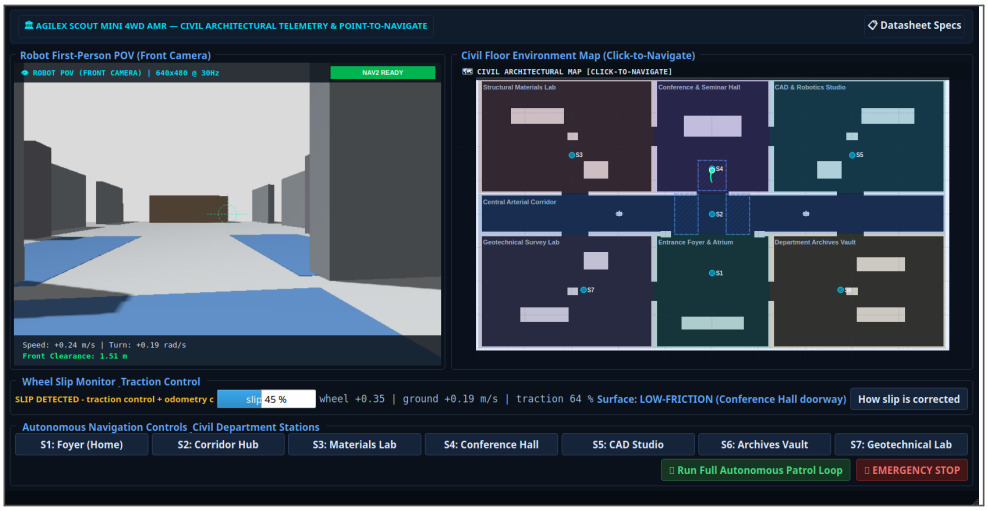


Fig. 16. The same patch with the correction ON: slip detected at 45 %; the wheel command is cut to 0.35 m/s against a ground speed of 0.19 m/s and traction is reduced to 64 %.

Click-to-navigate by the shortest path. The robot chose long routes between rooms and a click often needed several tries, on both floors. The earlier fix split each click into door-aligned waypoints (door, corridor lane, door, target); this forced detours along the corridor, and a new click could be ignored while an old route was still running. The Ground Station now sends one NavigateToPose goal and the Smac 2D planner finds the shortest path through the doorways. Each click increments a route counter (route_id); an older route notices that it was superseded and stops without cancelling the new goal, so one click is enough. The clicked point is moved to the nearest cell at least 0.55 m from obstacles so that Nav2 does not reject or abort it, and a goal the robot has stayed within 0.30 m of for 6 s is cancelled and counted as reached. The emergency stop cancels the route. In Nav2, cost_travel_multiplier is 1.0 and yaw_goal_tolerance 3.14 rad, so the planner minimises distance and the robot does not spin to a final heading.

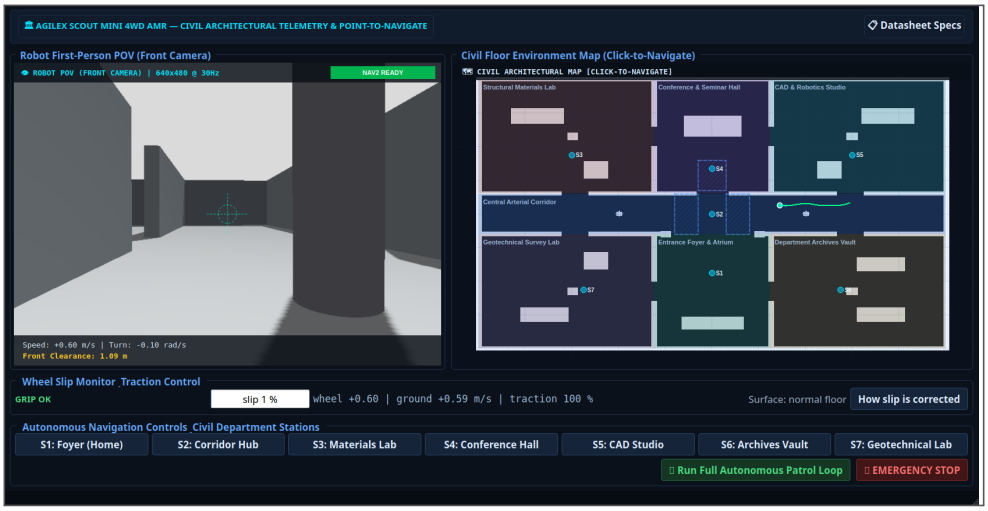


Fig. 17. Ground Station with a goal in the Conference Hall; the green line is the path planned by Nav2.

7. Algorithms Used

This section gives, in pseudo-code, the algorithms of the second review as implemented in slip_control.py, civil_interactive_nav.py and civil_nav2_mission.py, followed by the existing algorithms that were used unchanged. Symbols: v_w wheel speed from odometry, v_{est} estimated ground speed, a_f filtered forward acceleration, a_b its bias, ω_z IMU yaw rate, dt the sample period.

A. Slip estimation and correction (slip_control)

Algorithm 1. Ground-speed estimation (on every IMU sample, 100 Hz)

```
a   <- forward IMU acceleration x sign - gravity leak (from the IMU orientation)
a_f <- a_f + 0.3 (a - a_f)                          # low-pass filter
quiet <- |v_w| < 0.02 and |a_f - a_b| < 0.05 and |w_z| < 0.03
quiet_t <- quiet_t + dt if quiet else 0
if quiet_t > 0.3:                                    # zero-velocity update
    a_b <- a_b + 0.02 (a_f - a_b);  v_est <- v_w
else:
    v_est <- v_est + (a_f - a_b) dt                  # integrate the IMU
    if not slip:                                     # complementary pull to the wheels
        d    <- v_w - v_est
        gate <- exp(-(d/0.05)^2) * exp(-((a_w - a_f + a_b)/0.4)^2)
        v_est <- v_est + min(3.0 gate dt, 1) d
if body steady (|a_f - a_b| < 0.08) for 0.5 s and slip and not quiet:
    v_est <- v_est + min(3 dt, 1) (v_w - v_est)      # latch breaker (IMU drift)
v_peak <- max(|v_w|, v_peak - 0.3 dt)                # energy bound
v_est  <- clamp(v_est, -(v_peak + 0.05), v_peak + 0.05)
```

The gate stops a spinning wheel from pulling the estimate toward the wheel speed: the pull is strong only when the wheel and IMU agree about speed and acceleration.

Algorithm 2. Slip detection with hysteresis and debounce

```
slip_speed <- v_w - v_est
ratio      <- clamp(slip_speed / max(|v_w|, 0.2), -1, 1)       # published as /slip/ratio
if not slip:
    if |slip_speed| > 0.15:   cand_t <- cand_t + dt
    else:                     cand_t <- 0
    if cand_t >= 0.05:        slip <- True                      # SLIP DETECTED
else if |slip_speed| < 0.06:
    slip <- False;  t_clear <- now                              # GRIP RECOVERED
```

The enter threshold (0.15 m/s) is higher than the exit threshold (0.06 m/s), which prevents the flag from chattering.

Algorithm 3. Traction limiter and heading hold (control step, 50 Hz)

```
c_v, c_w <- command from Nav2 (0 if older than 0.5 s)
limiting <- slip or (now - t_clear) < 0.6
target_v <- c_v
if limiting:
    target_v <- clamp(c_v, v_est - 0.12, v_est + 0.12)             # wheels may lead the ground by 0.12
    target_v <- clamp(target_v, min(c_v, out_v), max(c_v, out_v))  # never above request or output
out_v <- slew(out_v, target_v, 4.0 m/s^2, dt)
scale <- |out_v| / |c_v|                                           # /slip/traction_scale
w_target <- c_w + clamp(0.4 (c_w - w_z), -0.6, 0.6)                # yaw-rate feedback
out_w <- slew(out_w, w_target, 6.0 (3.0 if limiting) rad/s^2, dt)
publish (out_v, out_w) on /cmd_vel_safe
```

This is traction control in the sense of a car: while the wheels spin, the speed command is held close to the true ground speed so that the tyres return to peak grip, and the heading is held by feeding back the gyro.

Algorithm 4. Sideways slip estimation (IMU lateral channel)

```
a_y <- lateral IMU acceleration (low-pass, bias a_yb)
if quiet_t > 0.3:  a_yb <- a_yb + 0.02 (a_y - a_yb);  v_y <- 0
else:
    v_y <- v_y + (a_y - a_yb - w_z v_est) dt        # remove the centripetal term
    v_y <- v_y - min(2.0 exp(-(v_y/0.06)^2) dt, 1) v_y   # decays to 0 when the tyres grip
    v_y <- clamp(v_y, -1.5, 1.5)
```

Algorithm 5. Odometry correction and corrected pose (on every /odom message)

```
d <- v_w - v_est
if enabled and |d| > 0.04:
    v <- v_w - sign(d) (|d| - 0.04);  if v v_w < 0: v <- 0     # never flips direction
if enabled and |v_y| > 0.03:  v_y <- v_y - sign(v_y) 0.03
yaw <- yaw + w_z dt                                            # IMU yaw, not wheel yaw
x   <- x + (v cos yaw - v_y sin yaw) dt
y   <- y + (v sin yaw + v_y cos yaw) dt
publish /odom_corrected (v, v_y, w_z, pose); the EKF uses it instead of /odom
```

With the correction switched off (slip_correction:=false) the node publishes the raw speed and passes the command through unchanged, which gave the comparison runs.

B. Goal and mission management

Algorithm 6. Click-to-navigate with pre-emption (Ground Station)

```
on map click (x, y):
    route_id <- route_id + 1;  my_id <- route_id
    cancel the active NavigateToPose goal
    (gx, gy) <- nearest free cell >= 0.55 m from any obstacle to (x, y)    # goal snap
    send ONE NavigateToPose goal (gx, gy, heading = direction of travel)
    while goal active and my_id == route_id:
        if distance(robot, goal) < 0.30 m for 6 s:  cancel goal; count as reached
    if my_id != route_id:  stop quietly (a newer click owns the robot)
on emergency stop:  route_id <- route_id + 1;  cancel goal;  publish zero velocity
```

The planner (Smac 2D) chooses the shortest path through the doorways; the counter ensures that one click is enough, even if a route is still running.

Algorithm 7. Patrol leg with closed-loop verification (civil_nav2_mission)

```
for each leg (name, x, y, heading, station) in MISSION:                # 20 legs
    for attempt in 1..3:
        send NavigateToPose(x, y, heading);  wait up to 120 s
        err_amcl  <- distance(AMCL pose, goal);   err_truth <- distance(Gazebo pose, goal)
        err_yaw   <- |heading error|
        ok <- status == SUCCEEDED and err_amcl <= 0.45 and err_truth <= 0.45
        if station: ok <- ok and err_yaw <= 40 deg
        if ok: break
        clear both costmaps;  retry
    if not ok: stop the run (leg failed)
    if station: wait 2 s
```

Algorithm 8. Route-efficiency measurement (nav_tour_test)

```
for each goal in TOUR:  snap goal >= 0.55 m from obstacles;  send NavigateToPose
    driven  <- sum of |displacement| of the map -> base_footprint transform at about 5 Hz
    optimal <- length of grid A* from the start of the leg (0.30 m clearance, 0.30 m cells)
    excess  <- driven / optimal - 1
```

8. How to Run

All commands are run from a terminal on Ubuntu 24.04 with ROS 2 Jazzy and Gazebo Harmonic installed. The Gazebo window, RViz2 and the Ground Station open on the desktop; the launch files set ROS_DOMAIN_ID 19 and local-only discovery so that the simulation does not mix with other ROS 2 programs on the network.

A. Requirements and build

Table 11. Software required.

Requirement	Used for
ROS 2 Jazzy, ros_gz (bridge, sim), Gazebo Harmonic	Simulation and the topic bridge
navigation2, nav2_bringup, nav2_smac_planner	Nav2 navigation stack
robot_localization, slam_toolbox	EKF and mapping
python3-pyqt5, numpy, matplotlib	Ground Station and test scripts

```
git clone https://github.com/Saibts/AgileX_ScoutMini_JAZZY.git ~/agilex
cd ~/agilex
git checkout slip-simulation
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install     # symlink install: script edits take effect at once
source install/setup.bash
```

B. Normal floor with the Ground Station

```
cd ~/agilex
./run_civil_nav.sh        # stops stale processes, then starts Gazebo, Nav2, RViz2 and the Ground Station
```

Wait about 40 s until the robot appears in the map. Click a point on the Ground Station map to send one direct goal; click again to replace it; the red button stops the robot. The station buttons S1–S7 send the robot to the named rooms.

C. Slip floor, correction on or off

```
cd ~/agilex
./run_slip_demo.sh                          # slip world, correction ON, runs the patrol mission
./run_slip_demo.sh --no-mission            # same, without the automatic patrol (click goals yourself)
./run_slip_demo.sh slip_correction:=false  # correction OFF: monitor only, for comparison

# equivalent explicit command
W=$HOME/agilex/install/assem2_robot/share/assem2_robot/worlds
ros2 launch assem2_robot civil_simulation.launch.py use_nav:=true use_gui:=true \
        world:=$W/civil_world_slip.sdf slip_correction:=true
```

The Wheel Slip Monitor on the Ground Station shows the slip percentage, the wheel and ground speed, the traction scale and the surface under the robot. The three blue hatched boxes on the map are the low-friction patches. The slip topics can be read directly:

```
ros2 topic echo /slip/active            # True while slip is detected
ros2 topic echo /slip/ratio             # slip ratio, -1 .. 1
ros2 topic echo /slip/traction_scale    # 1.0 = no limiting
ros2 topic echo /slip/correction_enabled   # latched: is the correction ON?
```

D. Patrol test, tour and other tools

```
# patrol with verification (start the simulation first, in another terminal, then wait for localisation)
source ~/agilex/install/setup.bash
ros2 run assem2_robot civil_nav2_mission      # 20 legs; prints status, AMCL and ground-truth error

# route-efficiency tour (seven direct goals)
python3 ~/agilex/src/assem2_robot/scripts/nav_tour_test.py

# keyboard teleoperation and SLAM mapping (first-stage tools)
./run_teleop.sh
./run_civil_mapping.sh && ./save_civil_map.sh

# inspect the system
ros2 node list; ros2 topic list; rqt_graph; ros2 run tf2_tools view_frames
```

E. Headless runs

For repeated headless tests the simulation was started with use_gui:=false and the mission run from a second script, which started a fresh simulation for every run and stopped all processes at the end (Section 9).

9. Experimental method

A. Patrol test

The patrol has 20 goals (legs) through the Conference Hall, the CAD Studio and the Materials Lab and back to the corridor hub (Appendix B lists the legs). Four legs are stations (Conference Hall, CAD Studio, Materials Lab, home) where the robot waits 2 s. The heading of each goal is the direction of travel from the previous waypoint. Each run starts from a freshly launched simulation with AMCL initialised at (0, 0). A leg succeeds when Nav2 reports success and both the AMCL pose and the Gazebo ground-truth pose are within 0.45 m of the goal; **at the four station legs the heading must also be within 40° of the goal heading**, as in the original script, while at other legs the heading is only recorded. A failed leg is retried up to three times (costmaps cleared between attempts); a leg that fails all attempts ends the run; a leg times out after 120 s. Three conditions were run, three times each in the order normal, slip on, slip off (repeated): the normal floor; the slip floor with the correction on; and the slip floor with the correction off. Two earlier runs with the same script are also reported (Section 10-B). Everything ran headless on one computer. Times are wall-clock seconds from the first goal to the end of the run.

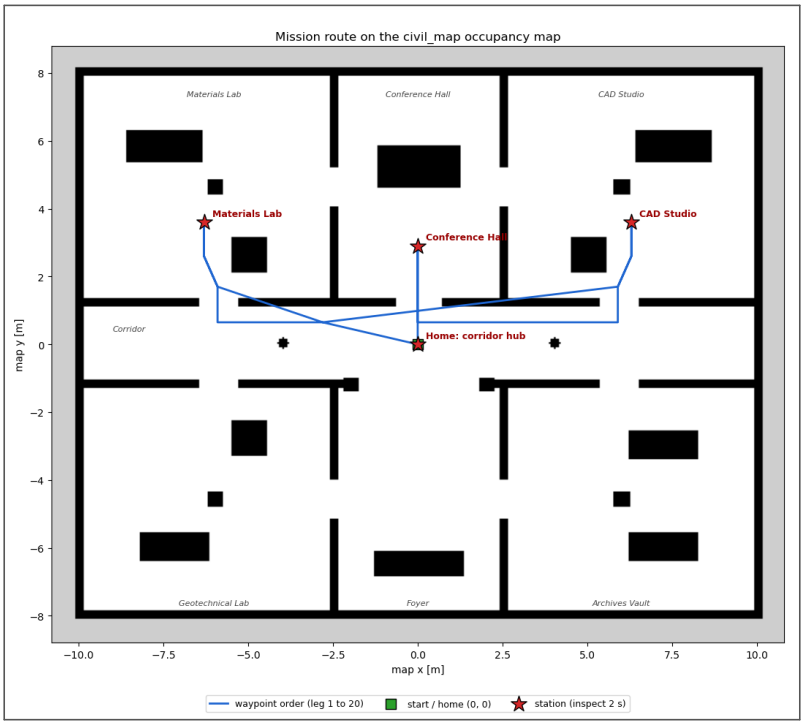


Fig. 18. The 20-leg patrol on the occupancy map. The blue line joins the waypoints in order and is not the measured path.

B. Route-efficiency tour

To measure how close driven routes are to the shortest possible, a tour of seven goals (Materials Lab, CAD Studio, Geotechnical Lab, Archives Vault, Conference Hall, Foyer and hub, in that order) was sent as single direct goals, each snapped to at least 0.55 m from obstacles as in the Ground Station. The driven distance is the sum of displacements of the map→base_footprint transform sampled at about 5 Hz; it is compared with the length of an A* path on the map grid with 0.30 m clearance from the start of each leg. Because the transform comes from the localisation system, the driven distance includes localisation noise. The tour was run once on the normal floor and once on the slip floor with the correction on.

10. Results

A. Normal floor (20-leg patrol)

Table 12. Ground-truth arrival error per leg, three normal-floor runs (bold = station legs).

Leg	Waypoint	Run 1 [m]	Run 2 [m]	Run 3 [m]	Mean AMCL error [m]	Max yaw error [°]
1	Conference Hall door	0.23	0.24	0.23	0.19	10
2	**Station: Conference Hall**	0.23	0.22	0.23	0.18	11
3	Conference Hall door (out)	0.22	0.21	0.24	0.17	20
4	Corridor hub	0.28	0.25	0.27	0.18	18
5	East lane (past post)	0.35	0.31	0.27	0.19	57
6	East lane (CAD door)	0.24	0.27	0.23	0.17	22
7	CAD Studio door (through)	0.25	0.26	0.25	0.19	2
8	CAD Studio (clear of rig)	0.20	0.21	0.20	0.18	2
9	**Station: CAD Studio**	0.16	0.18	0.20	0.18	36
10	CAD Studio (clear of rig)	0.22	0.17	0.20	0.17	27
11	CAD Studio door (through)	0.20	0.21	0.17	0.17	1
12	West lane (past post)	0.26	0.25	0.37	0.20	38
13	West lane (Materials door)	0.25	0.23	0.16	0.17	23
14	Materials Lab door (through)	0.24	0.24	0.23	0.18	5
15	Materials Lab (clear of press)	0.19	0.20	0.21	0.19	3
16	**Station: Materials Lab**	0.19	0.18	0.18	0.16	7
17	Materials Lab (clear of press)	0.31	0.28	0.30	0.19	3
18	Materials Lab door (through)	0.32	0.32	0.33	0.19	10
19	West lane (past post)	0.19	0.19	0.23	0.18	64
20	**Home: corridor hub**	0.21	0.19	0.19	0.18	13
	All legs	0.24	0.23	0.23	0.18	64

All three runs completed all 20 legs without any retry, in 198–251 s. Arrival errors against ground truth averaged 0.23 m and never exceeded 0.37 m (limit 0.45 m); the mean AMCL error was 0.18 m. The mean heading error at arrival was 15° and the maximum 64° (at a non-station leg, where heading is not checked). An additional normal-floor run made earlier with the same script also completed 20 of 20 legs (203 s, worst error 0.29 m).

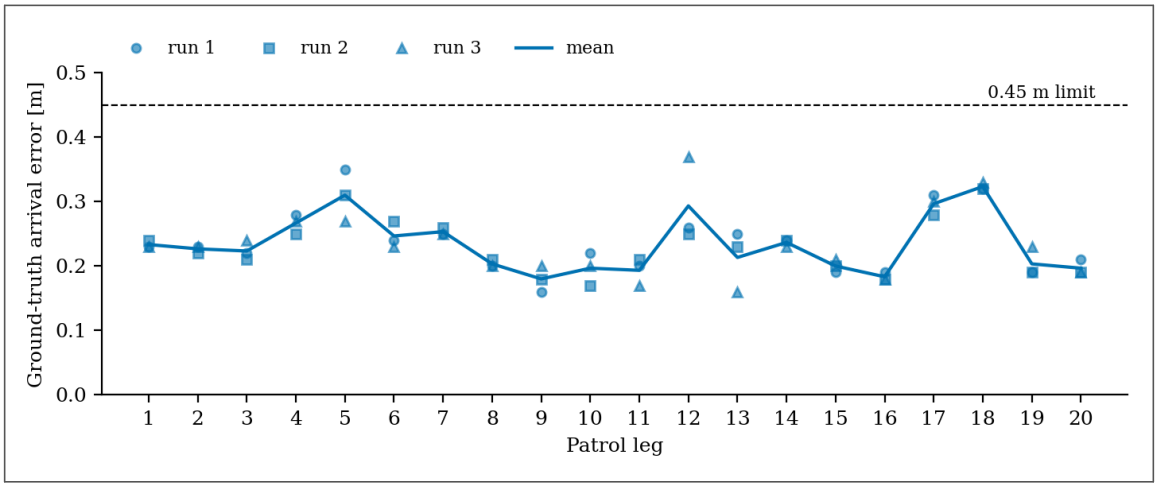


Fig. 19. Arrival error at each leg of the three normal-floor runs (dots) and their mean (line).

B. All runs of the final configuration

Table 13. Every run made with the final navigation configuration and the mission script of this report.

Condition	Runs made	Completed 20/20	Not completed	Wall time of completed runs [s]
Normal floor	4 (3 planned + 1 earlier)	4	–	198, 203, 210, 251
Slip floor, correction ON	4 (3 planned + 1 earlier)	3	1 stopped at leg 9 (CAD Studio station): heading error 45–46° against the 40° limit on all three attempts, position error 0.14 m	300, 335, 402
Slip floor, correction OFF	3	2	1 stopped at leg 4 after 44 s: the robot overshot the corridor hub by 0.61, 0.51 and 0.48 m (limit 0.45 m)	213, 243

The slip-floor run with the correction on that stopped at leg 9 is not a navigation failure: the robot was within 0.14 m of the goal, but Nav2 no longer controls the final heading (yaw tolerance 3.14 rad), so the station heading check is passed or failed by chance. Over the 38 station arrivals that passed, the heading error was up to 38° against the 40° limit, so the check is satisfied with little margin, and one of the four slip-floor-ON station arrivals at leg 9 failed it. This is a weakness of the test, not of the robot, and it is discussed in Section 12.

C. Slip floor: correction ON compared with OFF

Table 14. Correction OFF against ON in the planned nine-run set.

Measure	Correction OFF	Correction ON
Wheel command on a patch	Passes unchanged; wheels spin	Clamped to ground speed ± 0.12 m/s; traction scale reduced (64 % in Fig. 16)
Odometry	Raw wheel odometry over-reports	Slip-corrected /odom_corrected (vx and vy)
Planned runs completed	2 of 3 (legs reached: 3, 20, 20)	3 of 3 (legs reached: 20, 20, 20)
Mean / worst arrival error (truth)	0.28 / 0.45 m	0.22 / 0.33 m
Leg 4 (corridor hub), first arrival	Overshot the 0.45 m limit in 3 of 3 runs (0.46–0.61 m)	Within the limit in 3 of 3 runs
Retries (attempts repeated)	5 (all at leg 4)	5 (all at leg 12, aborted by Nav2)
Wall time of completed runs [s]	213, 243	300, 335, 402 (mean 346, against 220 on the normal floor)

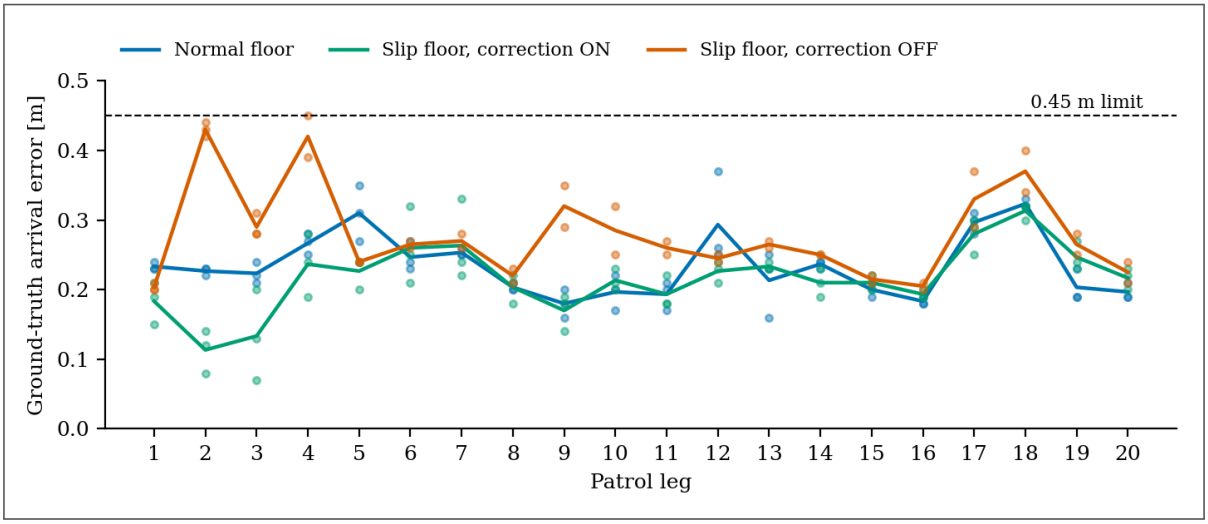


Fig. 20. Ground-truth arrival error at each leg in the three conditions. Lines are the mean over the runs that reached the leg; dots are single runs.

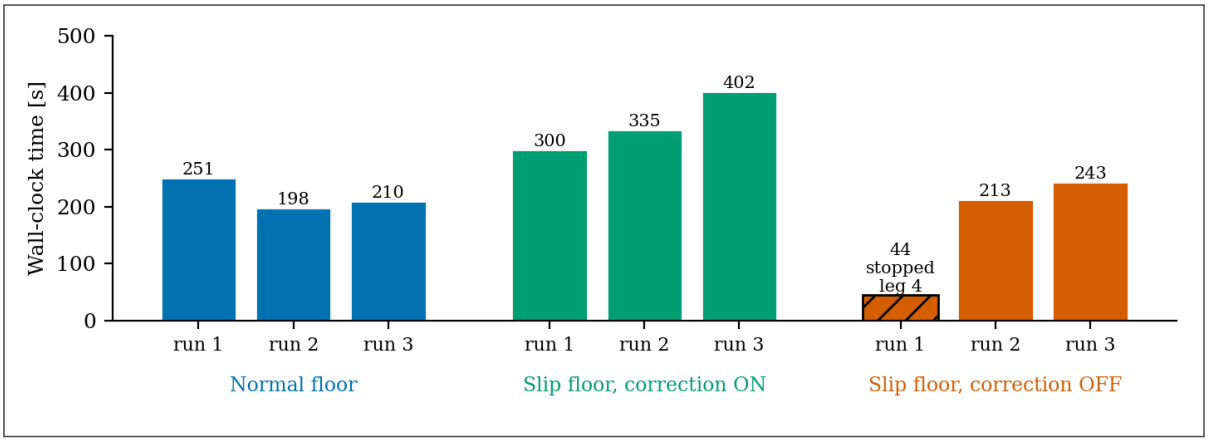


Fig. 21. Wall-clock time of the nine planned patrol runs; the hatched bar is the run with the correction off that stopped at leg 4.



Fig. 22. Legs completed (of 20) in each planned patrol run.

With the correction on, all three planned runs completed and the worst arrival error was lower (0.33 m against 0.45 m). The clearest difference is leg 4: with the correction off the robot slid past the corridor hub in all three runs after leaving the Conference Hall doorway patch. The correction costs time: the completed runs took 300–402 s against 198–251 s on the normal floor, and leg 12 (the west lane past the post, across both corridor patches) was aborted by Nav2 once or twice in every run before it succeeded on a retry; the cause was not investigated. With three runs per condition the difference in completion (3 of 3 against 2 of 3) is not statistically significant; the leg-4 overshoot, in all three uncorrected runs and in none of the others, is the more reliable signal.

D. Route efficiency

Table 15. Route-efficiency tour, normal floor (status 4 = Nav2 reported success).

Goal	Status	Driven [m]	Optimal [m]	Excess	Time [s]	Arrival error [m]
S3 Materials Lab	4	8.9	8.7	+2 %	19	0.19
S5 CAD Studio	4	13.3	12.8	+4 %	28	0.18
S7 Geotechnical Lab	4	16.3	15.9	+3 %	33	0.18
S6 Archives Vault	4	10.3	10.4	-1 %	23	0.18
S4 Conference Hall	4	10.8	10.8	+0 %	27	0.23
S1 Foyer	4	6.3	6.0	+4 %	18	0.18
S2 Hub	4	3.4	3.5	-2 %	12	0.19
Total		69.3	68.1	+1.8 %		

Table 16. Route-efficiency tour, slip floor with the correction on.

Goal	Status	Driven [m]	Optimal [m]	Excess	Time [s]	Arrival error [m]
S3 Materials Lab	4	8.8	8.7	+2 %	19	0.21
S5 CAD Studio	4	13.4	12.8	+4 %	32	0.18
S7 Geotechnical Lab	4	16.7	15.9	+6 %	67	0.17
S6 Archives Vault	4	10.3	10.3	+0 %	24	0.19
S4 Conference Hall	4	10.9	10.9	+0 %	27	0.18
S1 Foyer	4	6.1	6.0	+2 %	20	0.19
S2 Hub	4	3.3	3.4	-3 %	13	0.17
Total		69.5	68.0	+2.2 %		

Every goal was reached on both floors and the driven distance was between −3 % and +6 % of the grid reference per leg and +1.8 % (69.3 m against 68.1 m) over the whole normal-floor tour and +2.2 % on the slip floor. Negative values are possible because the reference keeps 0.30 m from obstacles and the measured distance comes from a sampled, filtered pose. Route length is therefore not the reason for the longer slip-floor patrol times. On the slip floor the Geotechnical Lab leg (S7) needed 67 s against 33 s for almost the same distance, so the extra time was spent driving slowly or pausing; this was not analysed.

E. Earlier slip results (recorded with earlier code and patch layouts)

Table 17. Earlier results, kept for reference. They came from the first recorded runs with earlier code and layouts and could not be repeated; they agree in direction with the new runs.

Measure	Correction OFF	Correction ON
Earlier 23-goal patrol, earlier layouts	3, 3, 3 and 21 legs (4 runs; none complete)	23, 21, 23, 23, 21, 21 and 5 (7 runs; 3 complete)
Final layout, single 23-goal run	3 of 23	23 of 23
Largest map-pose error in an earlier-layout capture run	3.9 m	1.6 m
Straight-line test on the μ = 0.02 floor	+1.12 m	+0.04 m

One further normal-floor run of the earlier 23-leg patrol, repeated on 10 November with the final Nav2 parameters, completed 23 of 23 legs in 193 s (worst truth error 0.32 m); it ran a stale installed copy of the script, which is why it had 23 legs.

11. Problems solved

Problems 1–16 were found while adding and testing slip; problems 17 and 18 were found on 10 November while re-measuring. Each is listed with its cause and fix, in the style of the first review.

Table 18. Problems met in the second review.

#	Problem	Root cause	Fix
1	Robot stopped at the edge of a patch	Raised tiles: a low-grip skid-steer robot cannot climb the edge; wheels spin and odometry keeps counting	Flush floor tiles with patch holes
2	Edits of slip_control.py had no effect	CMake installs a copy of the script	colcon build --symlink-install after every edit
3	Slip reported while stopped; correction stuck on	Estimate biased low after a stop (fusion pull, zero-velocity update while sliding, IMU drift)	Gated fusion, zero-velocity update only when quiet, latch breaker, energy bound
4	Wheel speed ran away to 1.44 m/s	clamp(cmd, v_est − m, v_est + m) could raise the command above the request	Target bounded between the request and the current output
5	Robot slid sideways with no change in odometry	Wheel odometry cannot see lateral slip	Lateral speed estimated from the IMU and fused in the EKF
6	Believed position drifted from the truth	Early version: wheel odometry over-reported on slip	Slip-corrected odometry in the EKF
7	The argument world:= was ignored	The launch file used a fixed world path	The launch file uses the world argument
8	Leftover Gazebo, RViz and static_transform_publisher processes corrupted TF	Processes of earlier runs were still running	Each run in its own process group, stopped by PID; checked before the next run
9	Clicking a goal sent the robot elsewhere or failed; several clicks needed	Door-aligned waypoints forced long routes and a new click was ignored during a route	Single direct goal with route_id pre-emption
10	Screenshots were black	Wayland does not allow screen capture by mss, PIL or the desktop D-Bus	Ground Station rendered off-screen with Qt; Gazebo through camera sensors; RViz2 window grabbed with xwd (XWayland)
11	Panel said “correction ACTIVE” with the correction off	The GUI could not tell the mode	Latched /slip/correction_enabled topic
12	Identical runs gave different results	Contact physics is not repeatable	Not fixed; all runs are reported
13	Robot slid sideways into a corridor post	Heading error carried onto ice, where the robot cannot steer; the post leaves about 0.02 m clearance	Corridor patches moved to the dry hub side and made full width
14	A second test simulation changed the results of the one already open	The launch file forced ROS_DOMAIN_ID 19	The launch file keeps an exported ROS_DOMAIN_ID; tests run in their own domain
15	Patrol stalled for 120 s at the east door, three times	A forced goal heading made the robot rotate in place (bang-bang ±0.6 rad/s); runs also used a stale installed script	Forced heading removed and script rebuilt; discarded runs not reported
16	Early theory (uneven grip) did not explain the slides	Real cause was heading error carried onto ice (problem 13)	Traced with recordings of true pose, command and path
17	**New.** *ros2 run civil_nav2_mission* ran an old 23-leg script	The installed copy is not a symlink and was not rebuilt after the source changed	colcon build --symlink-install; check the installed file against the source before runs
18	**New.** A run stopped at a station with the robot 0.14 m from the goal	Mission requires station heading ≤ 40° but Nav2 yaw tolerance is 3.14 rad, so heading is uncontrolled	Open: use a tighter yaw tolerance only on station goals, or check position only (13)

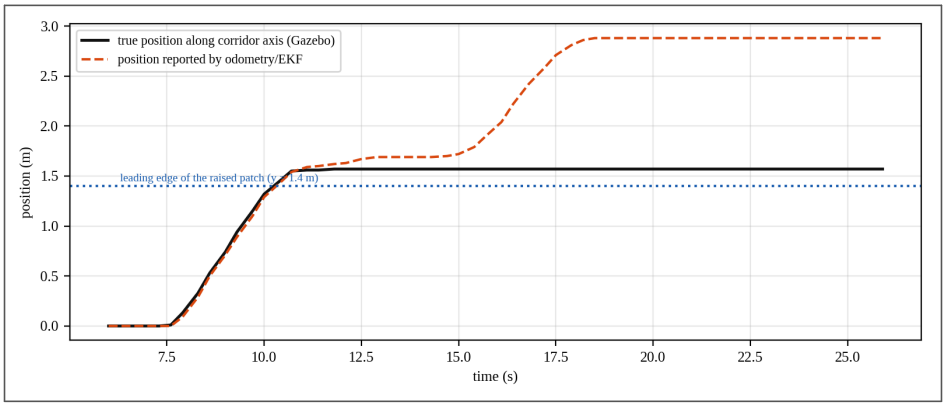


Fig. 23. Problem 1: with the raised patch edge the true robot (black) stops at y = 1.57 m against the edge, while its odometry (orange, dashed) keeps counting and drifts to 2.88 m.

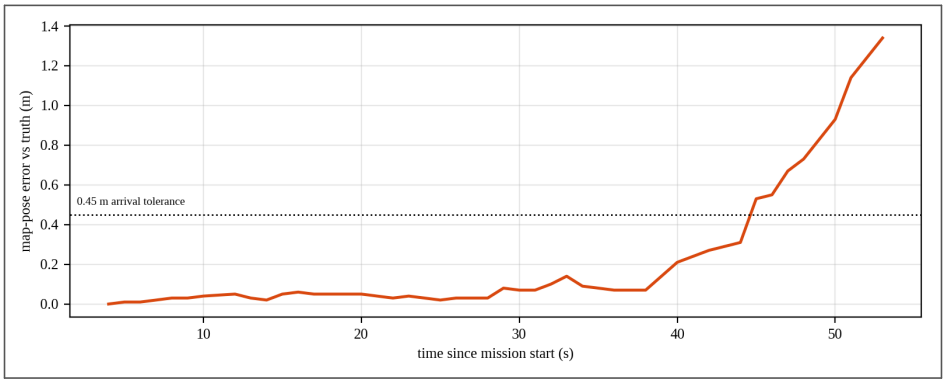


Fig. 24. Problem 6: map-pose error in an early run before the correction was added in its final form; it passes the 0.45 m tolerance at about 45 s and reaches 1.34 m at 53 s.

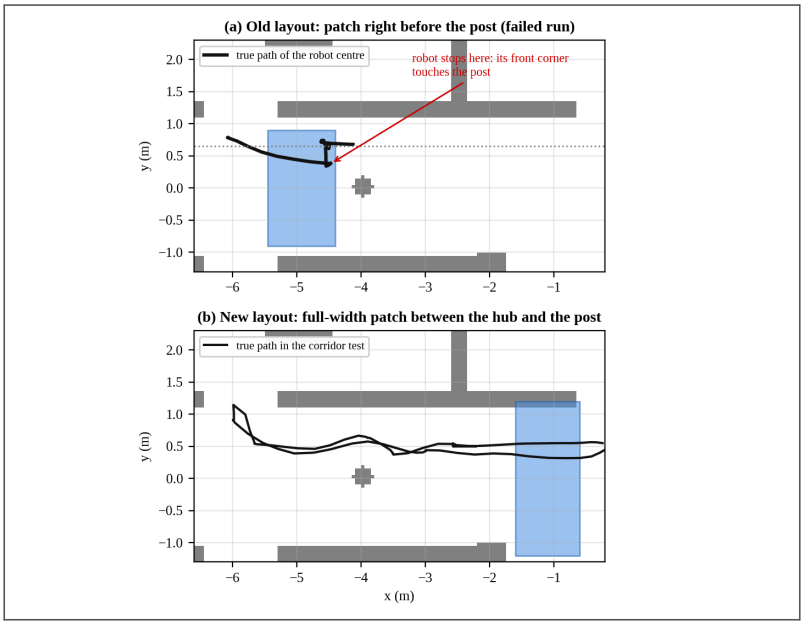


Fig. 25. Problem 13: old layout (a), the robot stops with a front corner on the post; final layout (b), the patch lies between the hub and the post and the true path stays clear.

12. Discussion and limitations

What the measurements show. On the normal floor the final configuration completed every run with ground-truth errors below 0.37 m. On the slip floor the correction kept all three planned runs complete, and its main visible effect was preventing the overshoot at leg 4. It slowed the patrol by about 1.6 times, and leg 12 needed retries in every run. The shortest-path change gave driven distances within about 2 % of the grid reference over a seven-goal tour.

Limitations.

- Three planned runs per condition (two extra runs on the normal and slip-ON floors); contact physics is not exactly repeatable. Completion rates (3 of 3 against 2 of 3) are not statistically significant.
- Simulation only. The slip patches are of our own design and their placement affects the results. The real robot's motor and battery behaviour is not modelled.
- **Heading at the stations is not controlled** (yaw tolerance 3.14 rad) yet the patrol checks it within 40°; one of the slip-ON runs failed that check at a position 0.14 m from the goal, and the passing stations reached up to 38°. The pass rate therefore understates how reliably the robot reaches the position.
- Tour distances come from the localisation pose, not ground truth. The extra time on slip legs (S7: 67 s against 33 s) and the leg-12 aborts were not investigated.
- Not evaluated in the second review: keyboard teleoperation, SLAM mapping and the 10 m obstacle arena (not re-run after the final changes), moving obstacles, and the real robot.

13. Conclusion and future work

The Scout Mini model navigates a 20-leg patrol on a normal floor in 198–251 s per run with arrival errors below 0.37 m. On a floor with three very slippery patches, the slip controller kept all three planned runs complete, while without it two of three completed and the robot slid past a goal in every run. Direct goal sending with route pre-emption replaced door-aligned routing, and the driven routes were within about 2 % of the shortest grid paths. The results come from a small number of simulated runs.

Future work: repeat the patrol many more times and over several patch layouts; investigate the leg-12 aborts; control or ignore the heading at stations (for example a tighter yaw tolerance only on station goals, or a position-only check); calibrate the odometry for skid-steer slip; use NavigateThroughPoses for smoother motion; monitor AMCL covariance instead of ground truth; and repeat the tests on the physical Scout Mini.

Appendix A. Key parameters

Table 19. Parameters used in the experiments.

Item	**Value (file)**
Planner	nav2_smac_planner::SmacPlanner2D, tolerance 0.25 m, max planning time 2 s, cost_travel_multiplier 1.0 (nav2_params.yaml)
Controller	RotationShimController (angular_dist_threshold 0.52 rad) → RegulatedPurePursuit, desired speed 0.60 m/s, look-ahead 0.25–0.60 m
Goal checker	xy_goal_tolerance 0.20 m, yaw_goal_tolerance 3.14 rad
Costmaps	resolution 0.05 m, inflation radius 0.55 m, footprint 0.66 × 0.70 m, local window 4 × 4 m (5 Hz), global 1 Hz
Velocity smoother	max 0.70 m/s and 1.0 rad/s; accel 1.0 m/s², 6.0 rad/s²; decel −1.5 m/s², −6.0 rad/s²; collision monitor polygon disabled
AMCL	1000–4000 particles, laser range 0.55–16 m, σ_{hit} 0.20, z_{hit} 0.9, z_{rand} 0.05, α_1 0.30 and $\alpha_{2..5}$ 0.10
EKF	20 Hz, 2D; odom vx and IMU yaw rate (ekf.yaml); /odom_corrected with slip control
slip_control	enter 0.15 m/s, exit 0.06 m/s, debounce 0.05 s, wheel lead 0.12 m/s, hold 0.6 s, linear accel 4.0 m/s² (0.8 declared, unused), angular accel 6.0 (3.0 while limiting) rad/s², yaw gain 0.4, yaw limit 0.6 rad/s, dead-bands 0.04 / 0.03 m/s
Slip patches	three flush patches, μ = 0.04 (civil_world_slip.sdf)
Mission checks	position within 0.45 m by AMCL and ground truth; station heading within 40°; 3 attempts per leg; 120 s per leg

The tour is scripts/nav_tour_test.py. The raw mission logs, tour logs and per-leg data (patrol.json) were committed to the slip-simulation branch under docs/experiment_logs/ (commit 6cf642a); the same commit holds the report-building scripts in docs/review_build/. They were later removed from the docs folder to keep only the final documents, and remain in the git history.

Appendix B. The 20 patrol legs

Table 20. Waypoints of the patrol (civil_nav2_mission.py).

Leg	Waypoint	x [m]	y [m]	Station (2 s, heading checked)
1	Conference Hall door	0.0	1.9	
2	STATION: Conference Hall	0.0	2.9	yes
3	Conference Hall door (out)	0.0	1.9	
4	Corridor hub	0.0	0.65	
5	East lane (past post)	2.8	0.65	
6	East lane (CAD door)	5.9	0.65	
7	CAD Studio door (through)	5.9	1.7	
8	CAD Studio (clear of rig)	6.3	2.6	
9	STATION: CAD Studio	6.3	3.6	yes
10	CAD Studio (clear of rig)	6.3	2.6	
11	CAD Studio door (through)	5.9	1.7	
12	West lane (past post)	-2.8	0.65	
13	West lane (Materials door)	-5.9	0.65	
14	Materials Lab door (through)	-5.9	1.7	
15	Materials Lab (clear of press)	-6.3	2.6	
16	STATION: Materials Lab	-6.3	3.6	yes
17	Materials Lab (clear of press)	-6.3	2.6	
18	Materials Lab door (through)	-5.9	1.7	
19	West lane (past post)	-2.8	0.65	
20	Home: corridor hub	0.0	0.0	yes